第7章 第11节 技术选型与实现

- 来源: https://class.imooc.com/lesson/5281#mid=104940
- 阶段: 阶段三: 【工具与协议】让AI连接世界
- 周: 第7周 MCP：模型上下文协议
- 模块: MCP通信协议与实战
- 课节: 图文 1-4 技术选型与实现

1-4 技术选型与实现

第7章 第11节 技术选型与实现

> [!TIP] 阅读指南 上一节做了需求分析和交互流程设计。本节进入技术选型阶段，解释为什么 Server 用 Node.js、Client 用 Python，以及项目的整体架构。

11.1 技术选型：为什么 Server 用 Node.js，Client 用 Python?

在开始动手之前，我们需要先回答一个关键问题：用什么语言来实现 MCP Server 和 Client?

MCP 的语言无关性

首先要明确一点：MCP 协议本身是语言无关的。

MCP 协议本身是语言无关的

回顾之前学习的通信机制：MCP 使用 JSON-RPC 2.0 作为通信协议，这是一个基于 JSON 的标准。只要一门语言能够：

- 处理 JSON 数据
- 通过 stdio 或 HTTP 进行通信

它就可以实现 MCP Server 或 Client。

理论上的可能性

```
MCP Server 可以用：
  • Node.js / TypeScript
  • Python
  • Go
  • Rust
  • Java
  • C#
  ... 任何语言都行

MCP Client 也可以用：
  • Python
  • Node.js
  • Go
  • Rust
  ... 任何语言都行
```

为什么官方推荐 Node.js 写 Server

虽然理论上可以用任何语言，但实际开发中，Node.js 是 MCP Server 的最佳选择：

1. 官方 SDK 支持最完善

Anthropic 官方提供的 @modelcontextprotocol/sdk 是 TypeScript/Node.js 实现的。使用官方 SDK 可以避免很多底层细节的坑。

2. 生态成熟

npm 上已经有大量现成的 MCP Server 可以参考：

- @modelcontextprotocol/server-filesystem - 文件系统操作
- @modelcontextprotocol/server-postgres - PostgreSQL 数据库
- @modelcontextprotocol/server-github - GitHub 集成

学习 Node.js 实现后，你可以快速理解和修改这些开源项目。

3. 异步 I/O 天然优势

MCP Server 需要处理大量的异步操作（读取文件、数据库查询、网络请求），Node.js 的事件循环模型非常适合这种场景。

为什么 Client 选择 Python

对于 AI Client 的开发，Python 是更好的选择：

Python 是更好的选择

1. Python是大模型 生态的事实标准

- 通义千问、OpenAI、Anthropic 等主流 大模型 服务都提供官方 Python SDK
- 数据处理库（pandas、numpy）非常成熟
- 机器学习工具链完善

2. 本书主语言

本书的其他章节（RAG、Agent、LangChain 等）都使用 Python，保持一致性可以降低学习成本。

3. Python 的 MCP Client SDK

虽然不如 Node.js 的 Server SDK 完善，但 Python 的 mcp 库已经足够好用：

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

# 非常简洁的 API
async with stdio_client(server_params) as (read, write):
    async with ClientSession(read, write) as session:
        # 调用 MCP 能力
```

新手友好的学习路径

如果你是新手，看到要同时用两种语言可能会有点担心，但不用慌：

对于 Node.js 部分（Server）：

- 我们的代码非常简洁（主程序不到 200 行）
- 主要是定义数据结构和注册函数
- 不需要深入的 JavaScript 知识
- 关键是理解 MCP 的概念，而非 Node.js 细节

对于 Python 部分（Client）：

- 如果你学过本书前面章节，Python 基础已经足够
- MCP Client 的 API 非常简单
- 重点是理解如何与 LLM 配合

学习建议：

现在大部分代码已经不需要一行一行手写了，AI 编程助手可以帮你完成大部分重复工作（即 Vibe-Coding）。但这不代表可以完全甩手——最基础的能力是读懂 AI 生成了什么。尤其是涉及企业级或生产环境代码时，纯粹的 Vibe-Coding 几乎无法胜任，你必须自己理解代码逻辑，才能判断 AI 给出的方案是否合理，有问题的部分才能精准修正。

好在门槛比过去低了很多：不需要记住每个 API 的参数细节，只需要看明白思路——这里的整体架构是什么样的、数据怎么流转、Server 和 Client 怎么配合。剩下的细节交给 AI 去填充就行。

11.2 实战项目说明

前面几节我们系统学习了 MCP 的理论知识——Resources、Tools、Prompts 三大核心能力，以及它们之间的通信机制。理论的最终目的是为了实践，现在让我们通过一个完整的项目来验证所学。

项目功能

我们将实现一个游戏装备计算助手，它能够：

游戏装备计算助手

1. 查询装备属性（Resources）

- 用户询问：“破晓之剑有什么属性？”
- 系统通过 Resources 读取装备数据，返回格式化的装备卡片

2. 计算战斗数值（Tools）

- 用户询问："这把剑能打多少伤害？"
- 系统调用伤害计算工具，执行公式并返回详细结果

3. 对比装备性能（Tools）

- 用户询问："破晓之剑和无锋重剑哪个更强？"
- 系统调用对比工具，给出数据分析和建议

4. 生成分析报告（Prompts）

- 用户需要："给我做一份装备深度分析"
- 系统使用 Prompt 模板，结合 LLM 生成专业报告

这个项目完整演示了 MCP 三大核心能力的实际应用。

项目目录结构

```
samples/chapter7/mcp_equipment/
├── README.md                    ← 完整技术文档（必读）
├── mcp-server/                  ← MCP Server 实现
│   ├── index.js                 ← Server 主程序
│   ├── package.json             ← Node.js 依赖配置
│   └── data/
│       └── equipments.json      ← 装备数据
├── ai-client/                   ← AI Client 实现
│   ├── main.py                  ← Client 主程序
│   └── requirements.txt         ← Python 依赖配置
└── .env.example                 ← 环境变量示例
```

关于项目的技术架构、源码讲解、环境配置和运行方法，请参考源码目录中的 README.md。这份文档结合了前面章节的理论知识，详细解析了每个模块的实现原理。

11.3 下一节预告

到这里，我们已经完成了一个从零开始的 MCP Server 开发。但每次需要一个新能力，都要从头写一遍 MCP Server 吗？答案当然是否定的。MCP 协议最大的价值不在于造轮子，而在于用轮子——整个开

源社区已经有大量现成的 MCP Server 可以直接使用。下一节将探讨如何快速找到并使用这些现成的 MCP Server，以及如何将自己开发的 MCP Server 发布出去。

11.4 ■ 学点英语

中文	English	音标	说明
技术选型	Technology Stack Selection	/tekˈnɑːlədʒi stæk sɪˈlekʃn/	根据场景选择合适的Server和Client开发语言的技术决策
语言无关性	Language Agnosticism	/ˈlæŋgwɪdʒ ˌægnɑːˈstɪsɪzəm/	MCP协议不限制实现语言，任何能处理JSON的语言都可使用
SDK 成熟度	SDK Maturity	/es diː keɪ məˈtʃʊrəti/	官方SDK对特定语言的开发工具和类型支持的完善程度
跨语言通信	Cross-Language Communication	/krɔːs ˈlæŋgwɪdʒ kəˌmjuːnɪˈkeɪʃn/	Server和Client用不同语言开发，通过标准协议互操作的能力